Real-world (compare your loop to the library's) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `agent_executor_comparison_practice.py`: now that you've built the loop by hand, LangChain's `create_agent` should look familiar — it's the same think → act → observe turns, pre-built, tested, and handling the same edge cases you just wrote yourself. Running your hand-built loop and `create_agent` on the same prompts side by side shows you exactly what the library does for you, and that it's doing the same thing, not something mysterious. **If not:** you'd either trust the library blindly, or rebuild it every time without knowing it already exists.

Needs `pip install langchain langgraph`.

Solution

```
# agent_executor_comparison_practice.py
from dataclasses import dataclass, field

from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.errors import GraphRecursionError


# Your own tools and ReAct loop from the build-the-loop exercise, inline so
# this file runs on its own.
@tool
def add(a: int, b: int) -> int:
    """Add two integers and return the result."""
    return a + b


ALL_TOOLS = [add]


class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list) -> None:
        self.steps = steps
```

```
        super().__init__(message)


@dataclass
class AgentResult:
    final_answer: str
    steps: list = field(default_factory=list)


def run_agent(task: str, max_iterations: int = 5) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools(ALL_TOOLS)
    messages = [{"role": "user", "content": task}]
    steps = []
    for step in range(max_iterations):
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return AgentResult(final_answer=response.content, steps=steps)
        for call in response.tool_calls:
            result = str(add.invoke(call["args"]))
            messages.append(
                {"role": "tool", "content": result,
                 "tool_call_id": call["id"]}
            )
            steps.append({"step": step, "tool": call["name"],
                          "result": result})
    raise MaxIterationsExceeded(
        f"No answer after {max_iterations} steps", steps
    )


PROMPTS = [
    "What's 5 + 7?",
    "What's the capital of France?",
    "Add 1 + 2, then add 3 to that.",
]

def count_library_tool_calls(messages: list) -> int:
    # how: every tool result in the run is a message with type "tool"
    count = 0
    for message in messages:
        if message.type == "tool":
            count = count + 1
    return count

@tool
```

```
def always_ask_again(city: str) -> str:
    """Look up the weather for a city. Always call it again to check."""
    return "Not sure yet - call this tool again."

# --- Compare tool-call counts: your loop vs the library's create_agent ---
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
agent = create_agent(model, tools=ALL_TOOLS)

for prompt in PROMPTS:
    mine = run_agent(prompt)
    library = agent.invoke({"messages": [{"role": "user", "content": prompt}]})
    library_calls = count_library_tool_calls(library["messages"])
    print(prompt)
    print(f"  mine:    {len(mine.steps)} tool calls")
    print(f"  library: {library_calls} tool calls")

# --- Both loops must stop a run that never ends — compare how ---
looping_agent = create_agent(model, tools=[always_ask_again])
loop_input = {"messages": [{"role": "user", "content": "Weather in Paris?"}]}
try:
    # recursion_limit counts graph steps — about 2 per turn
    looping_agent.invoke(loop_input, {"recursion_limit": 6})
except GraphRecursionError:
    print("library: GraphRecursionError at recursion_limit=6")
try:
    run_agent("Add 1 + 2, then add 3 to that.", max_iterations=1)
except MaxIterationsExceeded as e:
    print(f"mine:    MaxIterationsExceeded, {len(e.steps)} step logged")
```

Expected output (counts can vary a little between runs):

```
What's 5 + 7?
  mine:    1 tool calls
  library: 1 tool calls
What's the capital of France?
  mine:    0 tool calls
  library: 0 tool calls
Add 1 + 2, then add 3 to that.
  mine:    2 tool calls
  library: 2 tool calls
library: GraphRecursionError at recursion_limit=6
mine:    MaxIterationsExceeded, 1 step logged
```